Reviewer Pack — Minimal Rank of Matroidal Cycles Bounds Communication Comple…

Entry e74f224c3e4c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 18:07:33 UTC

Minimal Rank of Matroidal Cycles Bounds Communication Complexity for XOR

Entry ID: `e74f224c3e4c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 18:07:33 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Matroids) **Field B** (complexity object): Boolean Function Complexity: Circuit Complexity of XOR

Statement:

> For a matroid M associated with a Boolean function f, the minimal rank among all matroidal cycles in M is Θ(√n), where n is the number of variables in f. This implies that communication complexity for XOR is Ω(n^(1/2)).

Rationale (proposer's reasoning):

> Matroids have been used to represent combinatorial structures in computer science, and their cycles can be related to decision trees and communication protocols. A lower bound on the minimal rank of matroidal cycles could provide a new way to understand the complexity of XOR and potentially lead to more efficient algorithms for solving XOR-related problems.

Taxonomy category: `cg_kw_andreev` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8757822d30994f66`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the correlation coefficient between the minimal rank of matroidal cycles and communication complexity for XOR exceeds 0.5 across at least 100 random Boolean functions with n variables, where n ranges from 10 to 100.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	HITS	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "matroidal cycle rank" AND "Boolean function complexity" - "communication complexity" AND "matroid theory" - "XOR circuit complexity" AND "matroid minimal rank"

Top relevant hits considered: - [s2:10.1007/s00037-025-00276-5] Lifting Dichotomies - [s2:10.1137/S0895480103428338] On the Complexity of Some Enumeration Problems for Matroids

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def construct_matroid(f):
        n = len(f)
        matroid = []
        for i in range(1 << n):
            subset = [j for j in range(n) if (i & (1 << j))]
            if all(f[j] == f[subset[0]] for j in subset):
                matroid.append(subset)
        return matroid

    def communication_complexity(f):
        # Placeholder function to compute communication complexity
        # This is a dummy implementation and should be replaced with actual logic
        return len(f)  # Example: simply return the number of variables

    n = random.randint(10, 100)
    f = [random.choice([0, 1]) for _ in range(n)]

    matroid = construct_matroid(f)
    min_rank = min(len(cycle) for cycle in matroid) if matroid else float('inf')
    comm_complexity = communication_complexity(f)

    return {
        "metric_name": "min_rank",
        "metric_value": min_rank,
```

```
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": min_rank >= math.sqrt(n),
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 3 for i in range(5, 8

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means the required correlation coefficient could not be computed. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	28031
2	preregistration	ollama_remote	glm4:latest	0	0	14838
3	novelty	ollama_remote	glm4:latest	0	0	13824
4	novelty	ollama_remote	glm4:latest	0	0	12589
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14266
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11543
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	101033
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7032
9	judge	ollama_remote	glm4:latest	0	0	8941

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 212098 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e74f224c3e4c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e74f224c3e4c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e74f224c3e4c.tar.gz` (if generated)